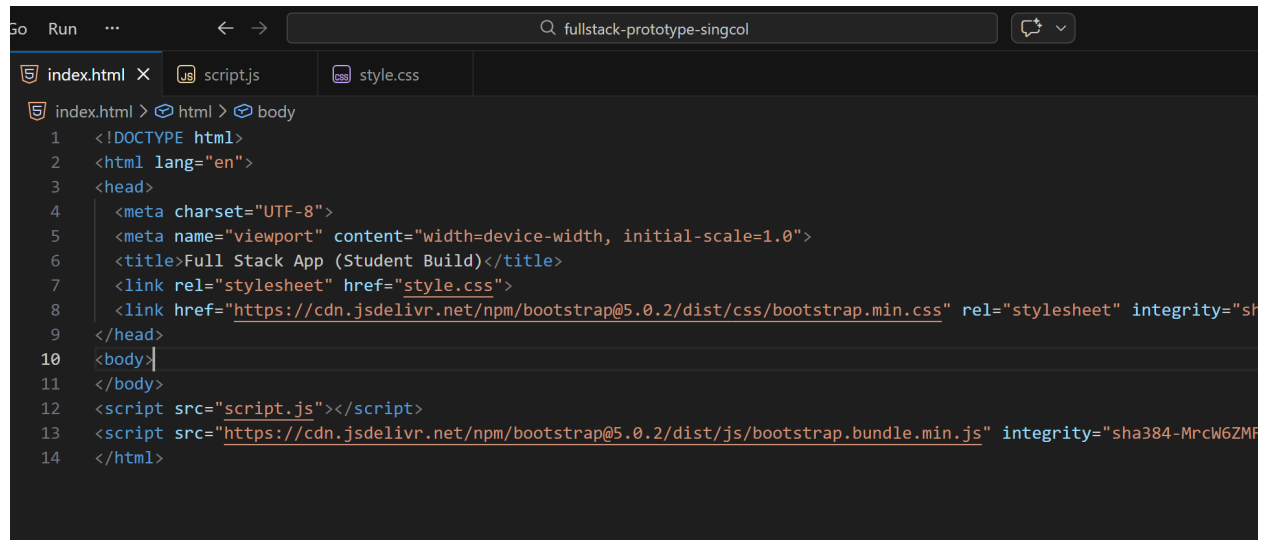


Phase 0: Setup & Planning

I've set up my project in a folder called `fullstack-prototype-singcol`. I created the basic files (`index.html`, `style.css`, and `script.js`) and added the standard HTML5 boilerplate. I also linked **Bootstrap 5** and my custom files to handle the design and logic.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Full Stack App (Student Build)</title>
7   <link rel="stylesheet" href="style.css">
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-MrcW6ZMF4L8aI/RZtNzPisK60JL2l+RU568wMG0l" crossorigin="anonymous">
9 </head>
10 <body>
11 </body>
12 <script src="script.js"></script>
13 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js" integrity="sha384-MrcW6ZMF4L8aI/RZtNzPisK60JL2l+RU568wMG0l" crossorigin="anonymous"></script>
14 </html>
```

Phase 1: Core Structure & Navigation

I learned bootstrap and wrote the code of all the pages in `index.html`.

1. Navigation Bar

Full-Stack App (Samantha Singcol)

Admin ▾

Welcome to Full-Stack App

A static prototype before backend integration. Simulates:

- Email registration + fake verification
- Login with JWT-like token simulation
- Role-based UI (Admin/User)
- CRUD for Employees, Departments, Requests

⚠ **Note:** All data is stored in `localStorage`. Refresh to reset.

Get Started →

I made the links to be admin-only: “Employees”, “Accounts”, “Departments” with .role-admin class

Full-Stack App (Samantha Singcol)

Admin ▾

My Profile

Admin User

Email: admin@example.com

Role: Admin

[Edit Profile](#)

2. Page Sections

- Home

Full-Stack App (Samantha Singcol)

Login

Register

Welcome to Full-Stack App

A static prototype before backend integration. Simulates:

- Email registration + fake verification
- Login with JWT-like token simulation
- Role-based UI (Admin/User)
- CRUD for Employees, Departments, Requests

⚠ **Note:** All data is stored in `localStorage`. Refresh to reset.

[Get Started →](#)

- Register

Full-Stack App (Samantha Singcol)

Login

Register

Register Account

First Name

Last Name

Email

Password

Sign Up

Cancel

- Verify Email

Full-Stack App (Samantha Singcol)

Login

Register

Verify Email

✓ A verification link has been sent to sam1@gmail.com.

For demo purposes, click below to simulate verification:

✓ Simulate Email Verification

Cancel

Full-Stack App (Samantha Singcol)

Login

Register

✓ Email verified! You may now log in.

Login

Email

Password

Login

Cancel

- Login

Full-Stack App (Samantha Singcol)

[Login](#)
[Register](#)

Login

Email

Password

[Login](#)[Cancel](#)

- Profile

Full-Stack App (Samantha Singcol)

Admin ▾

My Profile

Admin User

Email: admin@example.com

Role: Admin

[Edit Profile](#)

- Employees (admin)

Full-Stack App (Samantha Singcol) Admin ▾

Employees

+ Add Employee

ID	Email	Position	Dept	Actions
No employees.				

Add Employee

Employee ID

Account Email

Position

Department

Engineering ▾

Hire Date

mm/dd/yyyy

Save

Cancel

- Departments (admin)

Full-Stack App (Samantha Singcol) Admin ▾

Departments

+ Add Department

Name	Description	Actions
Engineering	Software team	EditDelete
HR	Human Resources	EditDelete

- Accounts (admin)

Full-Stack App (Samantha Singcol) Admin ▾

Accounts + Add Account

Name	Email	Role	Verified	Actions
Admin User	admin@example.com	Admin	✓	Edit Reset PW Delete

Add Account

First Name

Last Name

Email

Password

Leave blank to keep existing password when editing.

Role

User ▾

☐ Verified

Save

Cancel

- Requests

Full-Stack App (Samantha Singcol) User ▾

My Requests + New Request

You have no requests yet.

Create One

Full-Stack App (Samantha Singcol) User ▾

My Requests

You have no requests yet.

Create One

New Request

Type

Equipment ▾

Items

Item name1+

Submit Request

Cancel

+ New Request

3. CSS Rules** (in `style.css`):

```
css style.css > ...
17  body.not-authenticated .role-logged-in {
18      display: none;
19  }
20
21  body.authenticated .role-logged-out {
22      display: none;
23  }
24
25  .role-admin{
26      display: none;
27  }
28
29  body.is-admin .role-admin {
30      display: block;
31  }
32
33  body:not(.is-admin) .admin-only {
34      display: none !important;
35  }
36
37  .page {
38      display: none;
39  }
40
41  .page.active {
42      display: block;
43  }
```

4. Initial Body Class**: ``<body class="not-authenticated">``

```
30  <body class="not-authenticated">
```

Phase 2: Client-Side Routing

- A global variable: `let currentUser = null;`

```
// -----  
//  Global Auth State  
// -----  
  
let currentUser = null;  
  
// -----
```

- A function `navigateTo(hash)` that updates `window.location.hash`

```
// -----  
//  Navigation Helper  
// -----  
  
function navigateTo(hash) {  
  window.location.hash = hash;  
}  
  
// -----  
//  Core Router  
// -----  
  
function handleRouting() {  
  let hash = window.location.hash || "#/";  
  if (hash === "#") hash = "#/";  
  
  // Access-control guards  
  if (PROTECTED_ROUTES.has(hash) || ADMIN_ROUTES.has(hash)) {  
    if (!currentUser) {  
      window.location.hash = "#/login";  
      return;  
    }  
    if (ADMIN_ROUTES.has(hash) && currentUser.role !== "Admin") {  
      window.location.hash = "#/";  
      return;  
    }  
  }  
}
```

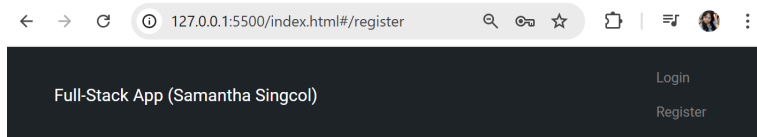

- A function `handleRouting()` that:
 - Reads current hash (e.g., `#/login`)
 - Hides all `.page` elements
 - Shows the matching page (e.g., `login-page`)
 - Redirects unauthenticated users away from protected routes (`#/profile`, etc.)
 - Blocks non-admins from admin routes

```
// Hide all pages
document
  .querySelectorAll(".page")
  .forEach((el) => el.classList.remove("active"));

// Show matched page
const pageId = ROUTES[hash];
if (pageId) {
  const target = document.getElementById(pageId);
  if (target) {
    target.classList.add("active");
  } else {
    console.warn(
      `[Router] No element with id="${pageId}" for hash "${hash}"`,
    );
    document.getElementById("home-page")?.classList.add("active");
  }
} else {
  document.getElementById("home-page")?.classList.add("active");
}

syncAuthClasses();
```

> 💡 Test: Type `#/register` in URL → Register form appears.



Register Account

First Name

Last Name

Email

Password

Sign Up

Cancel

Phase 3: Authentication System

A. Registration

```
// -----  
// A. Registration  
// -----  
  
function handleRegister() {  
  const firstName = document.getElementById("reg-firstname").value.trim();  
  const lastName = document.getElementById("reg-lastname").value.trim();  
  const email = document.getElementById("reg-email").value.trim().toLowerCase();  
  const password = document.getElementById("reg-password").value;  
  
  if (!firstName || !lastName || !email || !password) {  
    showError("register-page", "All fields are required.");  
    return;  
  }  
  
  if (password.length < 6) {  
    showError("register-page", "Password must be at least 6 characters.");  
    return;  
  }  
  
  const exists = window.db.accounts.find((a) => a.email === email);  
  if (exists) {  
    showError("register-page", "An account with that email already exists.");  
    return;  
  }  
  
  const newAccount = {  
    firstName,  
    lastName,  
    email,  
    password,  
    role: "User",  
    verified: false,  
  };  
  window.db.accounts.push(newAccount);  
  saveToStorage();  
  
  localStorage.setItem("unverified_email", email);  
  
  // Update verify page message before navigating  
  const verifyMsg = document.querySelector("#verify-page .alert-success");  
  if (verifyMsg)  
    verifyMsg.textContent = `✅ A verification link has been sent to ${email}.`;  
  navigateTo("#/verify");  
}
```

B. Email Verification (Simulated)

```
// -----  
// B. Email Verification (Simulated)  
// -----  
  
function handleVerify() {  
  const email = localStorage.getItem("unverified_email");  
  if (!email) {  
    showError("verify-page", "No pending verification. Please register first.");  
    return;  
  }  
  
  const account = window.db.accounts.find((a) => a.email === email);  
  if (!account) {  
    showError("verify-page", "Account not found. Please register again.");  
    return;  
  }  
  
  account.verified = true;  
  saveToStorage();  
  localStorage.removeItem("unverified_email");  
  
  localStorage.setItem("show_verified_banner", "1");  
  showSuccess("verify-page", "Email verified! Redirecting to login...");  
  setTimeout(() => navigateTo("#/login"), 1200);  
}
```

C. Login

```
// -----  
// C. Login  
// -----  
  
function handleLogin() {  
  const email = document  
    .getElementById("login-email")  
    .value.trim()  
    .toLowerCase();  
  const password = document.getElementById("login-password").value;  
  
  if (!email || !password) {  
    showError("login-page", "Please enter your email and password.");  
    return;  
  }  
  
  const account = window.db.accounts.find(  
    (a) => a.email === email && a.password === password,  
  );  
  
  if (!account) {  
    showError("login-page", "Invalid email or password.");  
    return;  
  }  
  
  if (!account.verified) {  
    showError("login-page", "Please verify your email before logging in.");  
    return;  
  }  
  
  localStorage.setItem("auth_token", account.email);  
  setAuthState(true, account);  
  navigateTo("#/profile");  
}
```

D. Auth State Management

```
//  
// PHASE 3 D. Auth State Management  
//  
  
function setAuthState(isAuth, user = null) {  
  currentUser = isAuth ? user : null;  
  syncAuthClasses();  
}  
  
function syncAuthClasses() {  
  document.body.classList.toggle("not-authenticated", !currentUser);  
  document.body.classList.toggle("authenticated", !!currentUser);  
  document.body.classList.toggle(  
    "is-admin",  
    !(currentUser && currentUser.role === "Admin"),  
  );  
  
  // Show/hide nav sections based on auth state  
  const loggedOutNav = document.querySelector(".role-logged-out");  
  const loggedInNav = document.querySelector(".role-logged-in");  
  
  if (loggedOutNav) loggedOutNav.classList.toggle("d-none", !currentUser);  
  if (loggedInNav) loggedInNav.classList.toggle("d-none", !currentUser);  
  
  // Update dropdown button label to reflect the user's actual role  
  const dropdownBtn = document.getElementById("adminDropdown");  
  if (dropdownBtn) {  
    dropdownBtn.textContent = currentUser ? currentUser.role : "Account";  
  }  
}
```

E. Logout

```
//  
// E. Logout  
//  
  
function handleLogout() {  
  localStorage.removeItem("auth_token");  
  setAuthState(false);  
  if (window.location.hash === "/" || window.location.hash === "#") {  
    handleRouting();  
  } else {  
    navigateTo("#/");  
  }  
}
```

Phase 4: Data Persistence with localStorage

- `STORAGE\_KEY = 'ipt\_demo\_v1'`

```
//  
// Storage Key  
// -----  
  
const STORAGE_KEY = "ipt_demo_v1";
```

- `loadFromStorage()`:

```
// -----  
// loadFromStorage()  
// Reads STORAGE_KEY from LocalStorage.  
// Falls back to seed data if missing or corrupt.  
// -----  
  
function loadFromStorage() {  
  try {  
    const raw = localStorage.getItem(STORAGE_KEY);  
    if (!raw) throw new Error("empty");  
    const parsed = JSON.parse(raw);  
    // Basic shape validation - must have accounts array  
    if (!Array.isArray(parsed.accounts)) throw new Error("corrupt");  
    window.db = parsed;  
  } catch {  
    window.db = getSeedData();  
    saveToStorage(); // persist the seed immediately  
  }  
}
```

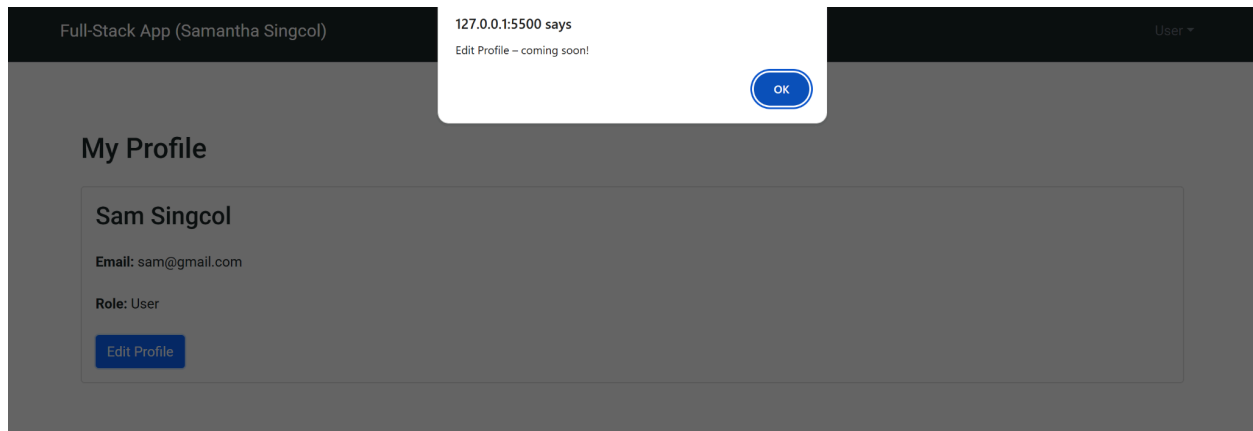
- `saveToStorage()`: stringifies `window.db` and saves

```
// -----  
// saveToStorage()  
// Stringifies window.db and writes to LocalStorage.  
// Call this after every create / update / delete.  
// -----  
  
function saveToStorage() {  
  localStorage.setItem(STORAGE_KEY, JSON.stringify(window.db));  
}
```

Phase 5: Profile Page

```
//  
// PHASE 5 – Profile Page  
//  
  
function renderProfile() {  
  if (!currentUser) return;  
  
  document.querySelector("#profile-page .card-title").textContent =  
    `${currentUser.firstName} ${currentUser.lastName}`;  
  
  document.querySelector("#profile-page .card-text").innerHTML =  
    `<strong>Email: </strong>${currentUser.email}`;  
  
  document.querySelector("#profile-page p:nth-of-type(2)").innerHTML =  
    `<strong>Role: </strong>${currentUser.role}`;  
  
  // Wire Edit button (once – guard against duplicate listeners)  
  const editBtn = document.querySelector("#profile-page .btn-outline-primary");  
  editBtn.replaceWith(editBtn.cloneNode(true)); // clone strips old listeners  
  document  
    .querySelector("#profile-page .btn-outline-primary")  
    .addEventListener("click", () => alert("Edit Profile 🕒 coming soon!"));  
}
```

- Shows “Edit Profile” button (can just show alert for now)



Phase 6: Admin Features (CRUD)

A. Accounts

```
//
// PHASE 6-A - Accounts CRUD
//

// Tracks which email is being edited (null = adding new)
let editAccountEmail = null;

function renderAccountsList(): void {
function renderAccountsList() {
const tbody = document.querySelector("#accounts-page table tbody");
if (!tbody) return;

if (window.db.accounts.length === 0) {
tbody.innerHTML = `<tr><td colspan="5" class="text-center py-3 bg-light">No accounts.</td></tr>`;
return;
}

tbody.innerHTML = window.db.accounts
  .map(
    (acc) => `
    <tr>
      <td>${acc.firstName} ${acc.lastName}</td>
      <td>${acc.email}</td>
      <td>${acc.role}</td>
      <td>${acc.verified ? "✔" : "-"}</td>
      <td>
        <button class="btn btn-outline-primary btn-sm me-1"
          onclick="openEditAccountModal('${acc.email}')">Edit</button>
        <button class="btn btn-outline-warning btn-sm me-1"
          onclick="openResetPwModal('${acc.email}')">Reset PW</button>
        <button class="btn btn-outline-danger btn-sm"
          onclick="openDeleteAccountModal('${acc.email}')">Delete</button>
      </td>
    </tr>
  `
  )
  .join("");
}

// — Reset PW Modal —

function openResetPwModal(email) {
document.getElementById("reset-pw-email-label").textContent = email;
document.getElementById("modal-new-password").value = "";
document.getElementById("resetPwModal").dataset.targetEmail = email;

removeAlert("reset-pw-modal-body");
new bootstrap.Modal(document.getElementById("resetPwModal")).show();
}

document.addEventListener("DOMContentLoaded", () => {
document
  .getElementById("modal-reset-pw-btn")
  .addEventListener("click", () => {
const email = document.getElementById("resetPwModal").dataset.targetEmail;
const newPw = document.getElementById("modal-new-password").value;
```

B. Departments

```
//
// PHASE 6-B - Departments CRUD
//

function renderDepartmentsList() {
const tbody = document.querySelector("#departments-page table tbody");
if (!tbody) return;

if (window.db.departments.length === 0) {
tbody.innerHTML = `<tr><td colspan="3" class="text-center py-3 bg-light">No departments.</td></tr>`;
return;
}

tbody.innerHTML = window.db.departments
  .map(
    (dept) => `
    <tr>
      <td class="ps-3">${dept.name}</td>
      <td>${dept.description}</td>
      <td>
        <button class="btn btn-outline-primary btn-sm me-1"
          onclick="editDepartment(${dept.id})">Edit</button>
        <button class="btn btn-outline-danger btn-sm"
          onclick="deleteDepartment(${dept.id})">Delete</button>
      </td>
    </tr>
  `
  )
  .join("");
}
```

C. Employees

```
// -----
// PHASE 6-C - Employees CRUD
// -----

let editingEmployeeId = null;

function renderEmployeesTable() {
  const tbody = document.querySelector("#employees-page table tbody");
  if (!tbody) return;

  if (window.db.employees.length === 0) {
    tbody.innerHTML = `<tr><td colspan="5" class="bg-light py-3 text-center">No employees.</td></tr>`;
  } else {
    tbody.innerHTML = window.db.employees
      .map((emp) => {
        const dept = window.db.departments.find((d) => d.id === emp.deptId);
        return `
        <tr>
          <td>${emp.employeeId}</td>
          <td>${emp.userEmail}</td>
          <td>${emp.position}</td>
          <td>${dept ? dept.name : "-"}</td>
          <td>
            <button class="btn btn-outline-primary btn-sm me-1"
              onclick="openEmployeeForm('${emp.employeeId}')">Edit</button>
            <button class="btn btn-outline-danger btn-sm"
              onclick="deleteEmployee('${emp.employeeId}')">Delete</button>
          </td>
        </tr>
        `;
      })
      .join('');
  }
};
```

Phase 7: User Requests

```
// -----
// PHASE 7 - My Requests
// -----

const STATUS_BADGE = {
  Pending: "warning",
  Approved: "success",
  Rejected: "danger",
};

function renderRequestsTable() {
  if (!currentUser) return;

  const myRequests = (window.db.requests || []).filter(
    (r) => r.employeeEmail === currentUser.email,
  );

  const emptyMsg = document.getElementById("requests-empty");
  const tableWrap = document.getElementById("requests-table-wrap");
  const tbody = document.querySelector("#requests-page table tbody");

  if (myRequests.length === 0) {
    if (emptyMsg) emptyMsg.classList.remove("d-none");
    if (tableWrap) tableWrap.classList.add("d-none");
    return;
  }

  if (emptyMsg) emptyMsg.classList.add("d-none");
  if (tableWrap) tableWrap.classList.remove("d-none");
}
```


Phase 8: Testing & Polish

```
// -----  
// PHASE 8 – Polish helpers  
// -----  
  
function clearForm(pageId) {  
  document.querySelectorAll(`#${pageId} input`).forEach((el) => {  
    if (el.type !== "checkbox") el.value = "";  
    else el.checked = false;  
  });  
  removeAlert(pageId);  
}  
  
// -----  
// DOMContentLoaded – Wire Everything Up  
// -----  
  
document.addEventListener("DOMContentLoaded", () => {  
  // 1. Load (or seed) data from localStorage  
  loadFromStorage();  
  
  // 2. Restore session before routing so guards work correctly  
  restoreSession();  
  
  // Set default hash silently (avoids spurious hashchange)  
  if (!window.location.hash || window.location.hash === "#") {  
    history.replaceState(null, "", "#/");  
  }  
})
```